Case Study:

File Sorter - Intelligent Desktop File Organization System

Kushagra Shrivastava

Personal Report

Author Note

This project was developed by **Kushagra Shrivastava** as a practical implementation of full-stack desktop application development, integrating Python, FastAPI, React, and Electron to automate file organization.

Abstract

Managing files manually becomes increasingly difficult as downloads, documents, media files, and project assets accumulate over time. Users often spend unnecessary time searching through cluttered folders and organizing files into appropriate categories.

File Sorter is a cross-platform desktop application that automatically organizes files into categorized folders based on their file extensions. The application provides an intuitive graphical interface while utilizing a high-performance Python backend to perform the sorting process efficiently.

The system combines **FastAPI**, **Electron**, and **React** to deliver a modern desktop experience with fast execution and a responsive user interface.

Problem Statement

Users frequently store all downloaded files in a single directory, resulting in:

- Cluttered folders
- Difficult file retrieval
- Reduced productivity
- Time wasted organizing files manually
- Increased risk of accidental deletion

Existing file managers either lack automation or require complicated configurations.

The objective was to create a lightweight desktop application capable of automatically organizing files with minimal user interaction.

Objectives

- Develop a desktop application for automatic file organization.
- Categorize files based on extensions.
- Provide a modern graphical interface.
- Ensure fast execution for large folders.
- Display sorting statistics after completion.
- Make the application executable without requiring Python installation.

Technology Used

Component	Technology
Backend	Python
API Framework	FastAPI
Frontend	React (Vite)
Desktop Framework	Electron
Styling	CSS
Packaging	Electron Builder
Backend Packaging	PyInstaller

Working

Step 1

The user selects a folder using the desktop interface.

Step 2

The React frontend sends an HTTP request to the FastAPI backend.

Step 3

The backend scans every file within the selected directory.

Step 4

Each file extension is classified into predefined categories.

Examples include:

- Images
- Videos
- Audio

- Documents
- Archives
- Executables
- Source Code
- Others

Step 5

If the corresponding category folder does not exist, it is automatically created.

Step 6

The file is safely moved into its destination folder.

Step 7

After completion, the backend returns:

- Total files processed
- Successfully moved files
- Failed files
- Processing time
- Category-wise statistics

Step 8

The frontend displays animated counters and sorting statistics.

Features

- Automatic file classification
- Drag-and-drop folder selection
- Folder browser
- Real-time processing status
- Category-wise statistics
- Responsive desktop interface
- Cross-platform desktop application
- Packaged backend executable
- Fast local processing
- No internet connection required

File Categories

Category	Examples
Images	jpg, png, gif, bmp
Videos	mp4, mkv, avi
Audio	mp3, wav
Documents	pdf, docx, txt
Archives	zip, rar, 7z
Executables	exe, msi
Code	py, cpp, java, js, html
Others	Unknown extensions

Backend Implementations

The backend was developed using **FastAPI** and **Python**.

Responsibilities include:

- Folder scanning
- File classification
- Folder creation
- File movement
- Error handling
- Runtime calculation
- JSON response generation

The backend exposes REST APIs that communicate with the Electron frontend over localhost.

Frontend Implementations

The frontend was developed using **React** with **Vite**.

It provides:

- Clean desktop interface
- Folder selection dialog
- Drag-and-drop support
- Animated statistics
- Loading indicators
- Success and error messages

The frontend communicates with the FastAPI backend using HTTP requests.

Desktop Integration

Electron was used to package the web application as a native desktop application. Responsibilities include:

- Creating desktop windows
- Launching the backend executable
- IPC communication
- Packaging the application
- Native file dialogs

Challenges

During development several practical issues were encountered:

- Packaging the FastAPI backend as an executable.
- Communication between Electron and the backend.
- Managing backend startup before the frontend initialized.
- Correctly packaging backend resources using Electron Builder.
- Handling file permissions during sorting.
- Managing relative and packaged resource paths.
- Synchronizing frontend loading with backend availability.

These challenges improved the application's reliability and deployment process.

Results

- Organizes files automatically.
- Creates folders dynamically.
- Categorizes files accurately.
- Displays detailed sorting statistics.
- Operates completely offline.
- Provides a user-friendly desktop experience.

Advantages

- Easy to use
- Fast execution
- Lightweight
- Offline functionality
- Reduces manual effort
- Modern user interface
- Easily extendable
- Platform independent through Electron

Limitations

- Classification is currently based only on file extensions.
- No recursive folder sorting.
- Does not classify files using AI or content analysis.

Future Enhancements

Potential improvements include:

- AI-based file classification using content recognition.
- Duplicate file removal.
- Automatic background monitoring of selected folders.
- User-defined custom categories.
- Cloud storage integration.
- File search functionality.
- Dark mode and theme customization.
- Undo operation for recently sorted files.
- Scheduled automatic sorting.

Conclusions

File Sorter demonstrates how modern desktop technologies can be combined to solve a common file management problem efficiently. By integrating Python for backend processing, FastAPI for communication, React for a responsive interface, and Electron for desktop deployment, the project delivers a practical and user-friendly solution for organizing files. The application significantly reduces manual effort, improves file accessibility, and provides a scalable foundation for future enhancements such as AI-based classification, automated monitoring, and advanced file management features.

References

- Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. https://git-scm.com/book/en/v2
- Electron. (2026). *Electron Documentation*. https://www.electronjs.org/docs/latest
- FastAPI. (2026). *FastAPI Documentation*. https://fastapi.tiangolo.com/
- Microsoft. (2026). *Windows File System Documentation*. https://learn.microsoft.com/windows/win32/fileio/file-management
- PyInstaller Development Team. (2026). *PyInstaller Documentation*. https://pyinstaller.org/
- React. (2026). *React Documentation*. https://react.dev/
- Python Software Foundation. (2026). *Python 3 Documentation*. https://docs.python.org/3/
- Vite. (2026). *Vite Guide*. https://vite.dev/guide/

Footnotes

[1] All software, libraries, and documentation referenced in this case study were accessed during the development of the project in July 2026. Product names and trademarks are the property of their respective owners.